传智杯 · Vibe Coding 挑战赛

机房故障智能诊断助手

对话式排查 · AI 驱动 · 面向高校机房运维

作品名称	机房故障智能诊断助手
选题方向	场景化轻量 RAG 知识库
技术栈	Python · Flask · DeepSeek API · 原生前端
当前版本	v1.0.0
代码仓库	github.com/liyishuo1/dsh-fault-diagnosis
作者	李一硕

本文档由项目开发过程实时记录整理而成

目录

一、系统架构

1.1 整体结构

1.2 模块职责

1.3 数据存储设计

1.4 如何约束 AI 生成的代码

二、核心技术实现

2.1 对话式逐层排查

2.2 轻量 RAG 知识库

2.3 知识库自动沉淀

2.4 设备状态四态模型

2.5 并发写入保护

2.6 课表：从教务表格到「哪间空着」

2.7 运行总览：判定「能不能上课」

2.8 桌面化：把同一个应用装进窗口

2.9 界面性能：两个被「看着还行」掩盖的问题

三、意图控制策略

3.1 系统提示词设计

3.2 上下文是怎么拼出来的

3.3 输出是怎么被约束的

3.4 提示词的迭代过程

四、工程质量保障

4.1 自动化测试

4.2 持续集成

4.3 典型缺陷

五、可复现性论证

5.1 环境与依赖

5.2 复现步骤

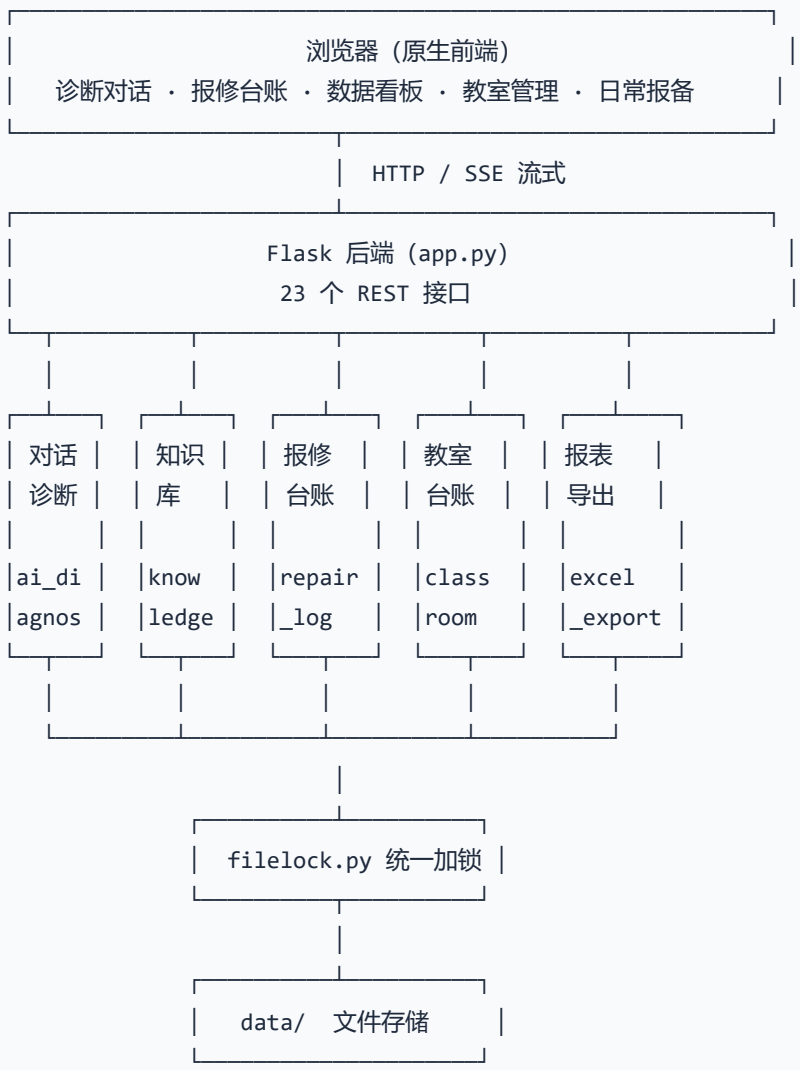
5.3 演进链路

六、AI 协作过程

> **这份文档只讲"怎么做"。** 项目是什么、解决什么问题、有哪些亮点，见同目录的 作品介绍文档 ； 设计取舍见 设计取舍.md ； 逐版迭代与缺陷修复见 迭代日志.md 。

一、系统架构

1.1 整体结构



1.2 模块职责

模块	行数	职责
`ai_diagnosis.py`	305	对话式诊断；维护会话历史；解析 AI 标记；数量解析
`knowledge.py`	494	知识库检索（轻量 RAG）；分类推断；自动沉淀
`repair_log.py`	430	报修台账 CRUD；状态机；数量统计；CSV 迁移
`classroom.py`	300	教学楼 / 教室两级台账；四态统计
`excel_export.py`	420	面向领导的一页纸报表（单表汇总）
`daily_task.py`	216	日常报备待办清单
`filelock.py`	86	跨平台文件锁（fcntl / msvcrt）
`app.py`	780	Flask 后端；23 个接口；SSE 流式输出

分层原则：业务逻辑全部放在独立模块中，`app.py` 只做「接收请求 → 调用模块 → 返回 JSON」的转发，不包含业务规则。 这样同一套逻辑既能被 Web 界面调用，也能被命令行 `main.py` 调用。

1.3 数据存储设计

本项目**不使用数据库**，全部采用文件存储。这是刻意的选择：

> **为什么不用数据库：** 本工具部署在机房现场的单机上，数据量极小（几十条记录 / 几个文件）， 引入 SQLite 或 MySQL 会增加部署复杂度，却带不来实际收益。 更重要的是——**纯文件便于人工查看与直接编辑**， 故障现场往往需要管理员直接用记事本打开台账确认， 这是数据库做不到的。

文件	是否提交	用途
`data/fault_cases.json`	✅ 提交	手工整理的知识库（23 条案例）
`data/fault_cases_auto.json`	❌ 忽略	程序自动沉淀的案例（运行时数据）
`data/repair_records.csv`	❌ 忽略	报修台账（含真实教室号，不公开）
`data/classrooms.json`	❌ 忽略	教室与设备台账
`data/daily_tasks.json`	❌ 忽略	日常报备待办

> **知识库分两个文件的设计理由：** 手工整理的经验应当随仓库传递、可评审、可回滚； 而程序自动生成的数据属于运行时产物，不应污染提交记录。 两者分离后，程序只写 `_auto` 文件，永不覆盖手工知识库。

1.4 如何约束 AI 生成的代码

AI 能写代码，但它不知道这个项目长什么样。所以**约束不是靠提示词， 是靠工程结构** —— 让它写出来的东西只能落在对的地方。

约束	具体做法
分层不许跨	`src/app.py` 只做路由和参数校验； 业务判断在 `overview.py` / `fault_category.py`； 读写文件只在 `repair_log.py` 等数据模块。 路由里出现业务 if-else 一律打回
数据文件单一来源	所有路径走 `paths.py`，模块里不允许再出现 `os.path.join(__file__, ...)`
改动必须带测试	新增判定规则、改统计口径，先补用例再改实现。 没测试的改动不进主干
一次提交一件事	一次提交对应一个功能模块或一次修复， 不做「顺手把三个页面一起改了」
前端零依赖	不引框架、不引构建工具，避免 AI 顺手加一个 npm 工程

实际效果：这套结构下，AI 生成的代码 95% 落在函数内部， 很少改动文件边界；一旦它开始改架构，代码评审这一关就会被拦下来。

> **最容易失控的地方是「顺手重构」。** AI 有时会为了实现一个小需求，把不相关的模块一起改掉。 发现这种情况一律回退 —— 改动范围和需求不一致，本身就是信号。

二、核心技术实现

2.1 对话式逐层排查

传统做法是用户输入故障后，一次性输出「5 个可能原因 + 5 个大步骤」， 内容长达上千字。在故障现场，管理员根本看不完，也无法根据实际情况推进。

本作品改为**对话式逐层追问**：

```
[用户] 501 教室电脑打不开
[AI]   按开机键后，机箱指示灯亮吗？风扇转吗？

[用户] 按电源没反应，灯不亮
[AI]   检查一下机箱后面的电源线两头有没有插紧，插紧后再按开机键，有反应吗？

[用户] 换了电源线还是不行
[AI]   看一下机箱背面电源上的开关是不是被关到"O"那边了，拨到"I"再开机试试？

[用户] 开关是开的，灯也不亮
[AI]   把电脑插到旁边正常的机位上试试，能开机吗？

[用户] 换了机位就能开
[AI]   这间教室有几台是这个情况？

[用户] 有 3 台
[AI]   【诊断结束】
       原因：这几个机位的插座没电（供电故障），主机本身没问题。
       数量：3 台
       处理：先确认是不是这几路插座对应的空气开关跳闸了……
       【需要报修】
```

关键实现——维护完整对话历史，让 AI 记住排查过程：

```
class DiagnosisSession:
    def __init__(self):
        self.history = []          # 对话历史
        self.classroom = ''        # 教室号
        self.quantity = 1          # 损坏数量
        self.need_repair = False   # 是否需报修
        self.self_fix = False      # 是否可自修

    def send(self, user_input):
        self.history.append({"role": "user", "content": user_input})
        response = self.client.chat.completions.create(
            model=MODEL,
            messages=self.history,      # 带上完整历史
            temperature=0.3,
            max_tokens=300,             # 限制长度，强制精简
        )
        reply = response.choices[0].message.content.strip()
        self.history.append({"role": "assistant", "content": reply})
        return clean_markers(reply), self.finished, self.need_repair
```

2.2 轻量 RAG 知识库

早期做法：把 `fault_cases.json` 全部案例塞进 system 消息。 案例一多就浪费 token，无关案例还会干扰判断。

现在的做法：先用本地关键词加权检索找出最相关的 Top-3 案例，只注入这几条。

检索算法

权重	命中位置
×3.0	故障分类一致
×2.0	现象描述命中
×0.5	可能原因命中（权重压低，见下）
额外加成	命中关键特征词表（如"蓝屏""断网"给 3.0）

> **为什么「原因」权重只有 0.5：** 原因是排查要点，用词宽泛（如"交换机断电或故障"里的"断电"）， 容易与无关领域偶然重合。曾经出现过查「电脑自动关机」时， 因共享「断电」一词而误命中网络类案例的假阳性。 压低权重并增加相关性判断后解决。

同义词归一

用户说「整间教室断网」，知识库里写的是「无法上网」—— 两边指的是同一件事，但字面完全没有重合，检索得分是 0。

```
SYNONYMS = {
    '上网': ['断网', '上不了网', '无法上网', '不能上网', '连不上网'],
    '开机': ['开不了机', '打不开', '无法开机', '点不亮'],
    '黑屏': ['无显示', '没显示', '屏幕黑', '不显示', '没画面'],
    # ... 共 17 组
}

# 反向索引：说法 → 标准词（模块加载时构建一次）
_SYNONYM_LOOKUP = {}
for _std, _variants in SYNONYMS.items():
    _SYNONYM_LOOKUP[_std] = _std
    for _v in _variants:
        _SYNONYM_LOOKUP[_v] = _std
```

关键在于**必须在切分之前扫描原文**：用户说「教室上不了网」， 按标点切分只会得到 `教室上` 、 `室上不` 这类碎片， `上不了网` 根本不会被提取出来。

粒度一致性

另一个被实际使用暴露的问题：查询「空调漏水」（4 字短句）整段保留， 而案例症状是长句、被切成 3-gram（ `空调漏` 、 `调漏水` ）—— 两边粒度不一致，交集为空，导致**刚沉淀的新案例用关键词反而搜不到。**

修复：长短句统一产出 3-gram，匹配时放宽为互为子串即算命中。

2.3 知识库自动沉淀

诊断结束后，若结论为「自己修」（已报修的不沉淀，因为结论尚未验证）， 系统把现象与结论提取成结构化案例写回知识库，实现**越用越聪明。**

```
def save_case(symptom, conclusion, excluded=None, classroom=''):
    # 整段持锁：查重 → 生成编号 → 追加 必须是一个原子操作
    with file_lock(AUTO_FILE):
        return _save_case_locked(symptom, conclusion, excluded, classroom)


def _save_case_locked(symptom, conclusion, excluded, classroom):
    # 1) 去重：现象高度相似的不重复添加
    existing = search(symptom, top_n=1, min_score=8.0)
    if existing:
        return False, f'已有高度相似案例 {existing[0].get("id")}'

    # 2) 从结论抽取「原因」和「处理」
    reason = re.search(r'原因[：: ]\s*(.+)', conclusion)
    action = re.search(r'处理[：: ]\s*(.+)', conclusion)

    # 3) 推断分类、生成编号（在合并后的全量案例中避让）
    category = _infer_category(symptom + ' ' + conclusion)
    new_id = f'{prefix}-{max(nums) + 1:03d}'

    # 4) 只追加到自动沉淀文件，不碰手工文件
    _append_auto_case(new_case)
    return True, new_id
```

> **为什么整段都要持锁（而不是只锁最后一步）：** 最初的实现只给「追加」那一步加锁，查重和编号生成都在锁外。 并发沉淀时两个线程会读到相同的文件内容， 于是各自判断「没有重复」、各自算出同一个编号 —— 实测两条「外设故障」都拿到了 `DEV-001`。
>
> 编号是基于「当前全部案例」算出来的，只要这次读取发生在锁外， 拿到的就必然是过期快照，冲突只是时间问题。

分类推断的规则设计

分类规则按「越具体越靠前」排列，第一个命中的生效。 这里踩过一个坑：早期版本把「声音」作为音频类的关键词， 导致**「风扇声音特别大」被判成音频故障**（散热问题被当成音响问题）。

```
_CATEGORY_RULES = [
    # 外设类先于音频：'音响''耳机'本身就带"响/机"
    (['鼠标', '键盘', 'U盘', 'usb', '摄像头'], '外设故障'),
    (['投影', '投影仪'], '投影故障'),
    (['打印机', '卡纸'], '打印故障'),
    (['上网', '断网', '网络', '网线', '交换机'], '网络故障'),
    (['蓝屏', '死机', '卡顿', '系统', '重装', '病毒'], '系统故障'),
    (['显示', '黑屏', '花屏', '无信号', '显示器'], '显示故障'),
    # 开机类：补齐 关机/断电/重启/硬件部件
    (['开机', '打不开', '不通电', '关机', '断电', '重启',
      '电源', '主板', '内存', '硬盘', '显卡', '风扇', '散热'], '开机故障'),
    # 音频类放最后，且用具体词
    (['音响', '音箱', '耳机', '话筒', '麦克风', '没声音', '杂音'], '音频故障'),
]
```

2.4 设备状态四态模型

设备不是「好」和「坏」两态。实际运维里有第三种：**可以自己修**（驱动问题、线松了），不需要报修。所以模型是四态：

```
未报修  →  可自修  →  已报修  →  已解决
              ↑                    |
              └──── 重置 ──────────┘
```

这个模型决定了台账排序、统计口径和看板上的「待处理」数量 —— 「可自修」不算待处理，但也不能从列表里消失。

2.5 并发写入保护

Flask 默认多线程，两个请求可能同时「读-改-写」同一个文件， 后写的会覆盖先写的，直接丢数据。最终是**两层锁**：

```
第一层  进程内 threading.RLock      — 同进程多线程
第二层  跨进程文件锁 (fcntl/msvcrt)  — 多进程 (Web + CLI 同时跑)
```

三个必须守住的点

要点	不守会怎样
先加锁再读取	先读再加锁，读到的是过期内容，锁白加了
「检查-写入」整体持锁	`if not exists(F): open(F,'w')` 之间没锁， 而 `'w'` 是截断写 —— 后到的把先写好的清空
锁必须可重入	内层调用重复加系统锁会死锁

第二条是后来才暴露的：Windows 的加锁与线程调度更慢，恰好错开竞态窗口， 本地一直是绿的；CI 换到 Ubuntu 才炸出来。实测 60 轮并发首次创建， **47 轮丢数据。**修法是全程持锁 + 创建改用 `'x'` 模式。

2.6 课表：从教务表格到「哪间空着」

设计理由（为什么没有查询按钮、调课怎么排序）见 `设计取舍.md` 。 这里只说解析和判定上的技术点。

格子的解析必须用锚点

> 专业软件CAD/SU/(5-5节)1-8周/石嘉伟/专业软件CAD/SU-0002/选课人数：26

课程名和教学班名里都可能带斜杠，按 `/` 硬切会碎成 7 段。 所以用「`(N-N节)X-Y周` 」这一段当锚点，前后分别处理。 一个格子里还可能有两门课（不同周次交替），用换行分隔，都要提取。

文件格式**按文件头判断，不看后缀** —— 学校系统的后缀经常不准： OLE2 走 `xlrd` ，ZIP 走 `openpyxl` 。

作息时间必须单独存

课表里只有「第几节」，**没有几点到几点**，而「现在有没有课」需要真实时间。 所以单独存一份作息表，并提供按锚点生成 12 节的入口（三个段起点 + 单节时长 + 课间）。

空余查询必须区分三类

类别	含义
空闲	已导入课表，且这一节确实没排课
占用中	这一节有课，附课程 / 教师 / 人数
未导入课表	不知道有没有课

> **第三类绝对不能算成空闲。** 不知道有没有课就说「空着」，会把学生引到正在上课的教室去。 这是这一块最容易被写错的地方，测试里专门锁了这条。

教师姓名在出口处脱敏

脱敏放在**后端返回时**做，不是前端 —— 前端做的话真名仍在接口 JSON 里， 打开开发者工具就能看到。复姓（欧阳、司马）按两个字处理，避免变成「欧老师」。

2.7 运行总览：判定「能不能上课」

「坏了几台」回答不了运维每天要回答的问题，所以判定改成五条，命中任一即算受影响：

条件	判定依据
整间跳闸	未解决的设施故障，现象含 总闸 / 跳闸 / 停电 / 短路 / 漏电
整间断网	故障对象是教室设施，且提到 交换机 / 上联 / 路由器
教师机故障	故障出现在教师机 / 讲台
教学设备故障	故障对象是教室设施，且提到 投影 / 幕布 / 音响 / 中控
座位不够	可用台数 < 该节次选课人数（仅在这一节有课时判断）

边界是刻意划清的：投影、幕布、音响算（电脑不缺也没法演示）， 空调、灯管不算（课照样上，只是难受）。

预警必须跨天、必须合并

只扫「今天」时，周日晚上打开系统是整块空白，而明天早八的课此刻就受影响。 范围定为**今天 + 明天**，每天的周次单独算。

另外按「教室 + 日期」合并：否则一间教室坏了，第 1–12 节每节一条预警， 14 条里 5 条是同一间，真正要看的信息被自己的重复淹掉。

2.8 桌面化：把同一个应用装进窗口

Web 形态每次要先起服务再开浏览器，还留一个控制台窗口。 目标只是**换一种启动方式**：用 pywebview 起内嵌 WebView2 的原生窗口， 页面代码一行没改。

"打个包"这一步暴露了三个原本不存在的问题：

问题	处理
数据不能打进 exe	打包后 `__file__` 指向临时解包目录，数据写进去程序一关就清空。 拆成「只读资源」（`res_dir()`，跟着程序）与「可写数据」（`data_dir()`，exe 旁边的 `data/`）
单文件 exe 起不来	`Could not create temporary directory` —— 单文件每次运行要解压自己，被安全软件拦下。改用文件夹版
pywebview 偶发失败	`Failed to resolve Python.Runtime.Loader.Initialize`。 同一份程序换目录就好，原因未查清。加兜底：失败则用 Edge 的 `--app` 模式开窗口（须给独立 `--user-data-dir`， 否则新进程会交接给已有实例后立刻退出）

API Key 为什么要三处来源

打包给别人用，对方电脑没有环境变量，一诊断就报"未配置"。读取顺序： **进程环境变量** → `data/config.json` → **Windows 用户环境变量。**

第三条是实测逼出来的：用户设过 Key，但 `pythonw` 启动的进程读不到， 只能直接读注册表兜底。另外「清除」必须写一个显式 `disabled` 标记， 否则删掉配置文件里的 key 之后，注册表兜底又把它找回来。

安全边界

写 Key、打开目录、导出文件这类接口**只接受 127.0.0.1 的请求。** 默认只监听回环地址，本来就出不去；但万一有人把 host 改成 `0.0.0.0`， 没有这道检查，同网段任何人都能改你的 Key。

2.9 界面性能：两个被「看着还行」掩盖的问题

两个性能问题都不是"代码写得慢"，而是**用错了渲染路径。** 它们的共同点是：功能完全正常，只是慢。

问题一：动画背景 + 满屏 backdrop-filter

全站有 16 种元素用了背景模糊，共 35 处，其中包括 **每一条报修记录、课表表头的 12 个格子**、5 个 KPI 卡。 背景静态时完全看不出来——一旦背景动起来， 这些区域每一帧都要重新做一遍高斯模糊。

改成半透明纯色背景（不透明度从 0.72 提到 0.92）后， 肉眼几乎看不出区别，但这几十次每帧模糊全部消失。

问题二：画布不是画得慢，是合成慢

剩下的背景动画仍然卡。这次先测量再改，在打包后的窗口里插临时帧率表：

条件	整页帧率	背景每帧绘制耗时
背景限 20fps	145–182 fps	—
背景提到 60fps	**46–55 fps**	**1.0 ms**

> **绘制只要 1ms，帧率却掉了三分之二 —— 说明瓶颈不在绘制。** DPR=2 时这块全屏画布是 3414×2134（730 万像素）， 每秒要把这么大一层重新合成 60 次，把合成资源吃满了。

把画布固定为 1 倍分辨率（1707×1067，面积只剩 1/4），帧率恢复， 线条是柔和的渐隐色，肉眼几乎看不出差别。

> **这里有一课：为了省 CPU 把帧率限到 20fps，其实是拿"背景一卡一卡"换了流畅。** 在高刷屏上，20fps 的动画和 180fps 的界面放在一起， 用户的感受就是"掉帧"。**优化要看用户看到什么，不是看指标降了多少。**

三、意图控制策略

3.1 系统提示词设计

系统提示词是本项目最核心的设计。它的目标不是「让 AI 更聪明」， 而是**约束 AI 的行为边界**——把通用大模型改造成一个守规矩的运维助手。

你是一名高校机房的运维工程师，正在通过对话帮学生管理员排查设备故障。

核心规则（必须严格遵守）

1. 每次回复只做一件事：要么问一个问题，要么给出结论。
   绝不一次列出多个问题或多个步骤。

2. 教室号已知时绝不再问。只有完全不知道教室时才问。

3. 第一步问清具体表现。如果用户描述很模糊（如"电脑打不开"），
   先问清楚具体现象，不要跳到排查。

4. 提问要多样化，禁止套模板。
   不要每次都问"指示灯亮吗？风扇转吗？"
   根据故障类型选择不同的切入角度。

5. 根据回答推进：用户说"不行"就排除这个原因，进入下一个全新的可能。
   绝不重复问同一个问题。

6. 用户答非所问时：先简短确认他的信息，再把原来的问题换个更简单的说法。

7. 确定原因后先判断故障范围，再决定要不要问数量：
   · 个别机器故障 → 先问「这间教室有几台是这个情况？」
   · 范围性故障（整间断电、整间断网、总闸跳闸）→ 不要问「几台」，
     整间都不能用了，问"几台电脑"没有意义

8. 数量要记住，不要写错。绝不能写 0 台。

9. 涉及电气安全时，第一件事是止损，不是继续排查：
   总闸 / 空气开关跳闸，只允许让用户推一次。
   · 推一次就恢复 → 可以继续按普通故障处理
   · 推上去立刻又跳、或根本推不上去 → 立即停止，
     明确告诉用户「不要再反复推闸」，直接给结论并报修
   出现焦糊味、冒烟、火花、发烫、漏电迹象时同样立即停止。
   绝不能让学生反复合闸、拆配电箱、动墙内线路。

结论格式

【诊断结束】
原因：<一句话说明故障原因>
数量：
处理：<自己修 / 需要报修，并说明具体怎么做>

范围性故障省略「数量：」这一行。
故障出在教室级设施上（总闸、交换机、投影、空调）时，
额外加一行【教室设施】。

严禁

- 严禁一次列出多个排查步骤
- 严禁输出大段解释和原理
- 严禁重复已经排除的检查项
- 严禁使用「首先、其次、最后」这种结构化列点

安全规则是被真实使用逼出来的。 早期版本把「教室总闸跳闸，合闸即可」列为可自修项， 于是 AI 会让学生反复推闸。而短路状态下反复合闸可能产生电弧、甚至引起火灾 —— 对每天都在机房里的学生管理员来说，这是不该出现的提示。

现在的约束是：**总闸只允许推一次，推上去还跳就必须停手等电工。**

3.2 上下文是怎么拼出来的

「会问问题」不只看提示词，还看每一轮塞给模型什么。上下文由四块拼成：

块	内容	为什么要有
系统提示词	角色、排查原则、禁止项	定住行为边界，每轮都带
检索到的案例	字符 3-gram + 同义词匹配出的 Top-3 历史案例	让判断基于真实积累，而不是泛泛而谈
对话历史	本次会话已问过的问题与回答	避免重复问已经排除的检查项
设备状态	该教室已登记的未解决故障	已知「这台电源坏过」，就别再让它问一遍电源

关键取舍：检索结果只取 3 条。 塞满反而干扰判断 —— 早期把全部案例塞进上下文的做法， 案例一多就浪费 token，而且模型会开始"参考"不相关的条目。

3.3 输出是怎么被约束的

提示词只是第一层。只靠提示词，模型迟早会越界。真正稳住输出的是三层：

层	做法	拦住了什么
提示词	写死禁止项：一次只问一个问题、严禁列多个步骤、 严禁重复已排除项、严禁使用「首先其次最后」	一次给太多、结构化列点
代码	流式分句后做结构化解析；结论必须能解析出「原因 + 数量」才落账	格式不合规时不会污染台账
测试	固定用例验证分类与检索；分类误判会被测试抓住	行为回退

> **为什么必须有代码层和测试层：** 提示词是「请求」，不是「保证」。模型换了版本、温度调了、 上下文变长，行为都可能漂移。只有后两层能保证「昨天对的东西今天还对」。

3.4 提示词的迭代过程

不是一次调好的，而是**每发现一次不对就补一条规则。**真实的演进：

发现的问题	补的规则
一次问五个检查项，值班员不知道从哪下手	一次只问一个问题，问完等回答
回答它不爱听的信息时，模型会继续追问已经排除的项	严禁重复已经排除的检查项
输出了「首先…其次…最后」的清单，像说明书	严禁使用结构化列点
信息不足时自己编一个原因	不确定就说不确定，不要编
开始教用户拆机箱	只做判断，不给维修操作指引

每条规则背后都有一次具体的失败。规则不是设计出来的，是**被问题逼出来的。**

四、工程质量保障

4.1 自动化测试

项目包含 **371 个自动化测试**，覆盖业务逻辑的关键路径与历史缺陷：

测试文件	用例数	覆盖内容
`test_schedule.py`	56	课表解析、脱敏、周次推算、空余查询
`test_overview.py`	47	影响上课判定、断网、预警跨天、调课筛选
`test_quantity_and_status.py`	38	数量解析、状态迁移、批量登记
`test_knowledge.py`	37	检索质量、假阳性防护、自动沉淀
`test_fault_category.py`	36	故障细分类、反复故障阈值
其余 12 个文件	172	台账、并发锁、导出、路径解析、桌面启动器等

> **只测不变的契约，不测易变的实现。** 测「正常 + 损坏 = 设备总数」这条业务规则，而不是某个函数怎么写的—— 前者改代码时仍成立，后者稍作重构就误报。

4.2 持续集成

```yaml
name: tests
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        # Windows 也要测：文件锁在 Windows 走 msvcrt、Linux 走 fcntl，
        # 两条分支都必须验证
        os: [ubuntu-latest, windows-latest]
        python-version: ['3.10', '3.12']
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install -r requirements-dev.txt
      - run: python -m pytest tests -v
```

每次推送到 main 或提交 PR，自动在 **4 种环境组合**上跑一遍测试。

4.3 典型缺陷

开发中真实踩过并修复的缺陷，完整记录（成因、复现数据、修法）见 `迭代日志.md`。这里只留两条对**方法论**最有价值的：

一、异常被静默吞掉

`except Exception: return ""` 把「JSON 解析失败」伪装成了 「没有知识库」。程序照常运行、界面正常，但知识库从未被加载。

> **这是最危险的失败模式。** 崩溃会立刻暴露问题，静默失败只会让功能「看起来能用、实际少了一块」。

二、只在 Linux 暴露的「检查-写入」竞态

`if not os.path.exists(F): open(F, 'w')` —— 检查与写入之间没有锁， 而 `'w'` 是截断写。多线程同时首次调用时，后到的把先到者写的内容清空。

Windows 的加锁与线程调度更慢，恰好错开竞态窗口，本地一直是绿的。 实测 60 轮并发首次创建，**47 轮丢数据。**

> **判断标准：如果一条测试的结论取决于线程时序，那它测的就不是业务规则。** 这类缺陷很容易被当成「测试不稳定」而加个重试糊过去。

五、可复现性论证

5.1 环境与依赖

项目	要求
操作系统	Windows / Linux / macOS
Python	3.10 或更高（开发环境 3.15.0）
依赖库	核心 4 个：`openai` `flask` `openpyxl` `xlrd`（读教务导出的 .xls）；桌面窗口版另需 `pywebview`
数据库	无需数据库
前端构建	无需构建工具（原生 HTML/CSS/JS）
API Key	DeepSeek API Key（从环境变量读取）

5.2 复现步骤

```
# 1) 克隆仓库
git clone git@github.com:liyishuo1/dsh-fault-diagnosis.git
cd dsh-fault-diagnosis

# 2) 安装依赖（核心 4 个库）
pip install -r requirements.txt

# 3) 配置 API Key（不写入代码，从环境变量读取）
#    Windows PowerShell:
[Environment]::SetEnvironmentVariable("DEEPSEEK_API_KEY", "你的key", "User")
#    设置后需要新开一个终端

# 4) 启动
cd src
python app.py
# 浏览器访问 http://127.0.0.1:5000

# 5) 运行测试（可选，371 个用例）
pip install -r requirements-dev.txt
python -m pytest tests -v
```

> **复现时间：约 5 分钟。** 无需数据库、无需前端构建、无需 Docker。 这也是刻意保持「零额外依赖」的原因——降低复现门槛本身就是可复现性的一部分。

5.3 演进链路

代码仓库保留了完整的细粒度提交历史，共 **96 个提交**， 首次提交仅包含 `.gitignore`（不是一次性 initial commit）。 每次提交对应一个功能模块或一次修复：

版本	能力	多了什么
v0.1.0	**能查**	本地故障知识库 + 关键词匹配查询（决策树方案做出来验证后暂缓）
v0.2.0	**能问**	接入大模型 API、改为对话式逐层排查、Web 界面与报修台账
v0.3.0	**能记**	教室设备台账、RAG 检索增强与自动沉淀、并发写入保护、 自动化测试与 CI、导出 Excel
v0.4.0	**能管**	故障对象与细分类、反复故障分析、课表与调课建议、运行总览
v1.0.0	**能用**	桌面窗口版与 exe 打包、界面性能修复

版本号按语义化三段编：`MINOR` 表示内容大更新， `PATCH` 表示缺陷修复。 早先每个功能都 +0.1，一路排到 0.15.0，十五条看着像流水账、看不出主线； 现在收敛成 5 个里程碑，修 bug 单独记，不占版本号。

缺陷修复（不占版本号）

交付于	修复内容
v0.4.0	刷新按钮点了没反应；单间课表弹窗叉号关不掉； 投影等教学设备故障不产生预警
v1.0.0	桌面窗口里导出 Excel 无反应；课表卡在「读取中」； 窗口最大化后缩不小；下拉框在深色主题下白底白字

> **这张表只记"使用者能感知到"的缺陷。** 内部重构、改名、补测试都不进表 —— 否则修复列表会变成开发流水账， 和版本表一样失去信息量。

六、AI 协作过程

本项目全程使用 AI 辅助，但不是「让 AI 写完整代码」， 而是**人负责判断与验证，AI 负责实现与建议。**

环节	人	AI
需求分析	提出痛点、定义验收标准	给出数据结构与方案
编码实现	审查逻辑、要求逐段解释	生成代码、说明取舍
问题排查	**提供完整报错与复现步骤**	定位根因、给出修法
效果验证	**实际运行，发现与预期不符**	按反馈调整

> **AI 最大的风险不是写不出代码，是写错的时候看起来是对的。** 故障分类器曾把「电脑打不开」判成「显示故障」—— 诊断结论里的鉴别诊断（「换线后显示器正常」）被当成了关键词。 这类错误只能靠固定测试集发现，不能靠「看一眼觉得没问题」。

完整对话记录（含 Prompt 原文、AI 回复、我的纠偏）在同目录：

- `AI对话Prompt快照.md` —— 按阶段整理的真实提示词，过程证据
- `AI对话记录-关键片段.md` —— 对外展示用精简版
- `AI对话记录-完整版.md` —— 内部存档，不对外

这些记录由工具 `tools/extract_prompts.py` 从会话日志**自动导出**， 不是事后补写 —— 这是「过程可复现」最能自证的一种方式。
